

Classification of Music by Genre

Classical Machine Learning versus Neural Networks on the GTZAN Dataset

Brian Miller

TECH 27 — Machine Learning with Python — Summer 2026

1. Team Name and Team Members

- **Team name:** Not Applicable. Individual Project by Brian Miller
- **Team member:** Brian Miller (individual project)
- **Course:** TECH 27 — Machine Learning with Python, Summer 2026
- **Instructor:** Ishaani Priyadarshini
- **Submission date:** September 10, 2026

2. Objective

The objective of this project is to classify 30-second music clips into one of ten genres — blues, classical, country, disco, hip-hop, jazz, metal, pop, reggae and rock — and to compare how well different families of machine learning model perform the task.

The project compares ten models on the same held-out test set of 200 songs:

- **Part 1 — classical machine learning** on 58 engineered audio features: Support Vector Machine, Random Forest, k-Nearest Neighbors and Gradient Boosting, each trained with default settings and then tuned by grid search.
- **Part 2 — neural networks:** a feed-forward network on the same 58 features (baseline and tuned), and three convolutional networks on mel-spectrogram images — a CNN, the same CNN trained with two versions of data augmentation, and a CNN-LSTM hybrid.

Models are compared on performance (accuracy, precision, recall, F1, ROC AUC), training and tuning time, and model complexity. A central question is whether adding model capacity improves results on a dataset of this size.

3. Motivation

I love music and have worked on some other projects combining music and software. I am interested in recommendation systems, tagging digital files, searching and filtering music.

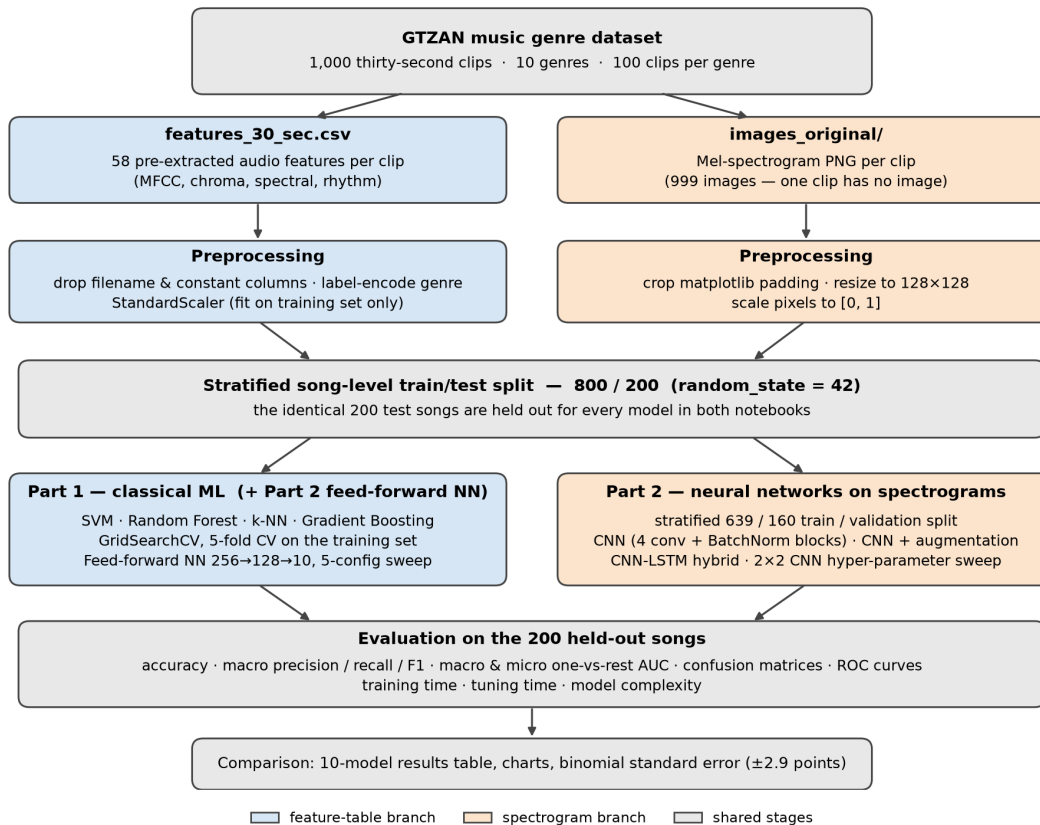
Because this class was about Machine Learning systems, and I found a data set about music genres, it seemed like an interesting project to work on.

4. Methodology

The project follows a single, controlled experimental design so that every model is directly comparable.

1. **One dataset, two representations.** GTZAN provides each clip both as a row of 58 pre-extracted audio features and as a rendered mel-spectrogram image. Classical models and the feed-forward network use the feature table; the convolutional models use the images.
2. **One fixed test set.** The 1,000 songs are split 800/200 with a stratified split (`random_state = 42`), giving exactly 20 test songs per genre. Section 2.2.1 of the Part 2 notebook reproduces this split for the spectrogram images by matching filenames, so *every model in the project is scored on the identical 200 songs*. One song has no spectrogram image, so the image training set has 799 songs.
3. **Baseline, then tune.** Every algorithm is first trained with sensible defaults, then tuned, and both results are recorded.
4. **Tuning never touches the test set.** Classical models are tuned with 5-fold cross-validation on the 800 training songs. Neural networks are tuned against a fixed, stratified 160-song validation set carved from the training data (16 songs per genre); the test set is used only once, for the final score.
5. **One evaluation function.** In Part 2 a single helper, `evaluate_keras_model`, produces every results row, so all neural models are scored identically and in the same schema as the Part 1 results.
6. **Statistical context.** Because the test set is only 200 songs, results are interpreted against the binomial standard error of a single accuracy estimate (about 2.9 percentage points) rather than read as a strict ranking.

5. Block Diagram



Project pipeline. The feature-table branch (blue) and spectrogram branch (orange) share one song-level train/test split and one evaluation procedure.

6. Tools and Techniques

6.1 Software

- **Python 3.12** in Jupyter notebooks
- **pandas** and **NumPy** for data handling
- **scikit-learn** for the classical models, preprocessing, grid search and all evaluation metrics
- **TensorFlow / Keras** for the feed-forward, convolutional and CNN-LSTM networks, and `tf.data` for the augmentation pipeline
- **Pillow** for loading and cropping spectrogram images
- **Matplotlib** for all plots

6.2 Data and Preprocessing

The GTZAN dataset (Tzanetakis and Cook, 2002) contains 1,000 thirty-second audio clips, 100 per genre, and is the most widely used benchmark for this task. The Kaggle distribution used here also supplies pre-computed features and pre-rendered spectrogram images.

Feature table. Each clip has 58 numeric features — means and variances of 20 MFCCs, chroma, spectral centroid, bandwidth and roll-off, zero-crossing rate, RMS energy, harmony, perceptual and tempo. Preprocessing consisted of dropping the filename column and any constant column, label-encoding the ten genres, and standardising every feature with `StandardScaler` fitted on the training set only, so no test-set statistics leak into training.

Spectrogram images. Each PNG was cropped to remove the matplotlib axes and padding (a 336×218 pixel plot area), resized to 128×128 , converted to three-channel RGB and scaled to the range $[0, 1]$.

Feature engineering — a note on scope. The audio features and spectrograms were both computed by the dataset’s creators; this project did not derive new features from the raw audio. The engineering work was in preparing and aligning the two representations: cleaning and scaling the feature table, cropping the images to their informative region, and — most importantly — reproducing the song-level split across both representations so that no song ever appears on both sides of it.

6.3 Models

Model	Input	Configuration
Support Vector Machine	58 features	RBF kernel; tuned C, kernel, gamma
Random Forest	58 features	tuned trees, depth, split and leaf sizes
k-Nearest Neighbors	58 features	tuned k, weighting, distance metric
Gradient Boosting	58 features	tuned learning rate, trees, depth
Feed-forward neural network	58 features	$256 \rightarrow 128 \rightarrow 10$, ReLU, dropout 0.3, Adam
Feed-forward NN (tuned)	58 features	best of a 5-configuration sweep
CNN	128×128 spectrogram	4 Conv2D + BatchNorm + MaxPool blocks (32–256 filters), global average pooling, dropout, softmax
CNN (fixed crop)	spectrogram	same CNN, augmentation that was unintentionally frozen across epochs
CNN (augmented)	spectrogram	same CNN, time-axis augmentation re-drawn every epoch
CNN-LSTM	spectrogram	3 conv blocks, then a bidirectional LSTM reading the time axis

Why these models. The four classical algorithms span the main families covered in the course — a margin-based classifier (SVM), a bagged tree ensemble (Random Forest), an instance-based method (k-NN) and a boosted tree ensemble (Gradient Boosting) — so

they test quite different inductive biases on the same features. The feed-forward network isolates the effect of switching to a neural model while keeping the input fixed. The CNN tests whether learning directly from the spectrogram image beats hand-engineered features, and the CNN-LSTM tests whether modelling the time axis as a sequence adds anything further. Augmentation and hyper-parameter tuning test whether the neural models can be improved without more data.

Most of the specific models evaluated were suggested by the class instructor.

7. Evaluation Metrics

Metric	What it measures	Why it was chosen
Accuracy	fraction of the 200 test songs labelled correctly	The classes are exactly balanced (20 test songs per genre), so accuracy is not distorted by class imbalance and is directly interpretable.
Macro precision	of the songs given a genre label, how many truly belong to it — averaged equally over genres	Shows when a model over-uses a label as a catch-all.
Macro recall	of the songs in each genre, how many were found — averaged equally over genres	Shows which genres a model misses.
Macro F1	harmonic mean of precision and recall	A single balanced summary of both error types.
Confusion matrix	counts of true versus predicted genre	Identifies <i>which</i> genres are confused with which.
One-vs-rest ROC curves	trade-off between true- and false-positive rate for each genre	Threshold-independent view of each genre.
Macro and micro AUC	area under the ROC curves, averaged per genre (macro) or pooled over all decisions (micro)	Measures ranking quality rather than only the top prediction — a model can rank the right genre highly yet rarely pick it first.
Training and tuning time	wall-clock seconds	The cost side of the comparison.
Complexity	decision nodes, support vectors × features, stored samples, or total parameters	Model size. These units differ by algorithm, so complexity is comparable only within a model family.

Macro averaging was used throughout so that every genre counts equally. AUC was included specifically because accuracy alone hides an important pattern in this data: for rock, the CNN's per-class AUC is 0.91 while its recall is only 0.60.

8. Success Criteria

- **Success:** at least one model reaches **80% or higher test accuracy** on the song-level held-out test set.
- **Secondary goal:** macro AUC of 0.95 or higher.

Against these, the tuned Random Forest (0.820 accuracy, 0.975 macro AUC) meets the primary criterion, and seven of the ten models meet the AUC criterion.

Success criteria were not fixed before the experiments, but during reporting. For the data set size of 200 songs, 0.820 is not statistical distinguishable from 0.80

9. Failure Criteria

- **Failure:** no model exceeds **50% test accuracy**, or any model performs no better than the 10% chance rate for ten balanced classes.

No model came close to failure: the weakest, k-Nearest Neighbors, reached 0.690 — about seven times the chance rate.

10. Project Timeline

Date	Milestone
August 19, 2026	Project proposal and dataset approved by instructor
August 12 - 20, 2026	GTZAN dataset downloaded and explored
August 27, 2026	Part 1 complete — four classical models trained, tuned and compared; results exported
September 8, 2026	Part 2 feed-forward network, CNN and CNN-LSTM built; batch-normalization bug found and fixed
September 8 - 9, 2026	Data augmentation experiments; augmentation bug diagnosed and fixed; CNN hyper-parameter tuning
September 7 - 10, 2026	Final clean top-to-bottom runs; notebook analysis revised against final results
September 10, 2026	Report and presentation completed
September 10, 2026	Presentation delivered

11. Implementation Details

11.1 Part 1 — classical models

Each of the four algorithms followed the same sequence: train with default settings and time the fit; predict the test set; compute training and test accuracy; compute macro precision, recall and F1; plot the confusion matrix and one-vs-rest ROC curves; compute macro and micro AUC; tune with GridSearchCV (5-fold cross-validation on the 800 training songs, accuracy scoring); then re-evaluate the tuned model on the test set. The tuned results were exported to `part1_results.csv` for Part 2.

Model	Grid searched	Combinations
SVM	$C \in \{0.1, 1, 10, 100\}$; kernel $\in \{\text{linear}, \text{rbf}, \text{poly}\}$; gamma $\in \{\text{scale}, \text{auto}, 0.001, 0.01, 0.1, 1\}$	72

Model	Grid searched	Combinations
Random Forest	trees $\in \{100, 200, 300\}$; max depth $\in \{5, 10, 20, \text{None}\}$; min split $\in \{2, 5, 10\}$; min leaf $\in \{1, 2, 4\}$	108
k-NN	k $\in \{3, 5, 7, 9, 11, 15\}$; weights $\in \{\text{uniform, distance}\}$; metric $\in \{\text{Euclidean, Manhattan}\}$	24
Gradient Boosting	learning rate $\in \{0.01, 0.1, 0.2, 0.3, 0.4\}$; trees $\in \{100, \dots, 500\}$; max depth $\in \{1, \dots, 5\}$	125

11.2 Part 2 — neural networks

All networks were built in Keras, compiled with the Adam optimiser and sparse categorical cross-entropy, trained with batch size 32, and seeded with `tf.keras.utils.set_random_seed(42)` inside each model-building function. Every network used early stopping with `restore_best_weights=True`, so the weights reported are those from the best validation epoch.

- **Feed-forward network:** $58 \rightarrow 256 \rightarrow 128 \rightarrow 10$ with ReLU and dropout 0.3 (49,290 parameters); learning rate $1e-3$; early stopping on validation loss, patience 15. The tuning sweep compared five configurations of layer sizes, dropout and learning rate on a fixed stratified validation set.
- **CNN:** four blocks of 3×3 convolution (32, 64, 128, 256 filters), batch normalization and 2×2 max pooling, then global average pooling, dropout 0.3 and a softmax layer — 392,906 total parameters (391,946 trainable). Learning rate $3e-4$; up to 100 epochs, patience 15. Batch-normalization momentum was set to 0.9 rather than the Keras default of 0.99 (see Section 17).
- **Augmentation:** a `tf.data` pipeline applied time-axis-only transformations to each training image — a random crop keeping 80–100% of the time axis, resized back to full width, plus two random time masks of up to 16 columns. The frequency axis was deliberately left untouched. Up to 200 epochs, patience 25.
- **CNN-LSTM:** three convolution blocks that pool the frequency axis aggressively but preserve time (final pool 4×1), giving a sequence of 32 time steps $\times$ 1,024 features, read by a bidirectional LSTM with 64 units; 653,002 parameters. Early stopping on validation accuracy, patience 20.
- **CNN tuning:** a 2×2 grid of dropout $\{0.3, 0.5\} \times$ learning rate $\{3e-4, 1e-3\}$, run with a reduced budget of 40 epochs and patience 8.

Hardware: Apple MacBook Pro, 2.3 GHz Quad-Core Intel Core i7, 32 GB Memory

12. Experimentation and Results

12.1 Part 1 — effect of tuning the classical models

Model	Default accuracy	Tuned accuracy	Tuned macro F1	Tuned macro AUC	Best parameters	Tuning time
SVM	0.735	0.765	0.764	0.944	RBF, C = 10, gamma = auto	11.9 s
Random Forest	0.775	0.820	0.818	0.975	300 trees, depth 20, min leaf 2	106.9 s
k-NN	0.675	0.690	0.690	0.905	k = 3, Manhattan, distance-weighted	0.4 s
Gradient Boosting	0.780	0.795	0.795	0.978	300 trees, depth 2, learning rate 0.2	3,234.8 s

Tuning improved every classical model’s accuracy, most for Random Forest (+4.5 points). It did not always improve AUC: tuned k-NN’s macro AUC fell from 0.920 to 0.905 even as its accuracy rose, because grid search optimised accuracy, not ranking quality. Random Forest and Gradient Boosting both fitted the training set almost perfectly (0.9988 training accuracy), leaving generalization gaps of 0.18 and 0.20.

12.2 Part 2 — neural networks

- **Feed-forward network:** 0.775 test accuracy against 0.921 training accuracy, trained in under 5 seconds. The tuning sweep selected the configuration already chosen by hand, and the retrained “tuned” model scored *lower* on test (0.755) — the five configurations were separated by at most four validation songs.
- **CNN:** 0.785 test accuracy, the best neural result, but with 0.998 training accuracy — a 21-point generalization gap — after 697 seconds of training.
- **CNN hyper-parameter tuning:** the 2×2 sweep again re-selected the hand-chosen configuration (dropout 0.3, learning rate $3e-4$). Learning rate was the only factor that clearly mattered (validation accuracy 0.72 at $3e-4$ against 0.63 at $1e-3$, averaged over dropout); the top two configurations were three validation songs apart.
- **Data augmentation:** slowed memorization — the augmented CNN needed about 40 epochs to approach the training accuracy the baseline reached in 18 — but did not improve test accuracy (0.760 and 0.740 for the corrected and frozen versions, against the baseline’s 0.785). Across three complete runs of the notebook the two augmentation variants swapped order, confirming that the difference between them is noise.
- **CNN-LSTM:** 0.700, the weakest neural model despite being the largest (653,002 parameters).

12.3 Combined comparison

Model	Family	Accuracy	Precision	Recall	F1	Macro AUC	Training time (s)	Tuning time (s)	Complexity
Random Forest	Classical	0.820	0.820	0.820	0.818	0.975	0.50	106.9	75,074 nodes
Gradient Boosting	Classical	0.795	0.802	0.795	0.795	0.978	9.97	3,234.8	2,076 nodes
CNN (spectrograms)	Neural	0.785	0.789	0.785	0.783	0.965	697.3	1,563.0	392,906 params
Feed-forward NN	Neural	0.775	0.779	0.775	0.771	0.962	4.64	—	49,290 params
SVM	Classical	0.765	0.769	0.765	0.764	0.944	0.03	11.9	38,454 SV × features
CNN (augmented)	Neural	0.760	0.767	0.760	0.761	0.964	1,556.7	—	392,906 params
Feed-forward NN (tuned)	Neural	0.755	0.760	0.755	0.753	0.958	4.53	36.3	49,290 params
CNN (fixed crop)	Neural	0.740	0.771	0.740	0.734	0.960	1,083.2	—	392,906 params
CNN-LSTM	Neural	0.700	0.701	0.700	0.693	0.940	587.4	—	653,002 params
k-NN	Classical	0.690	0.708	0.690	0.690	0.905	0.003	0.40	800 stored samples

Neural-network training times vary by several percent between runs of the notebook; figures shown are from the final run.

12.4 Key findings

1. **The best model is the simplest to train.** The tuned Random Forest reached 0.820 in half a second; the CNN took over a thousand times longer to land 3.5 points behind it.
2. **Most of the ranking is sampling noise.** With 200 test songs the standard error on an accuracy is about 2.9 points (95% margin ± 5.3). The top five models span 11 songs; only k-NN and the CNN-LSTM are clearly worse than the leader, by 26 and 24 songs.
3. **More capacity did not help.** Gradient Boosting with 2,076 decision nodes beat the 392,906-parameter CNN; the largest model, the CNN-LSTM, was the weakest neural network.
4. **Tuning, augmentation and architecture all failed to move the result** — three independent levers, one conclusion: the binding constraint is the number of independent training recordings, not the model.
5. **Per-genre structure is stable across models.** Classical music is among the easiest genres for every model; rock is the hardest and, for the CNN, also the most over-predicted label (predicted 21 times for 20 rock songs, only 12 correctly).

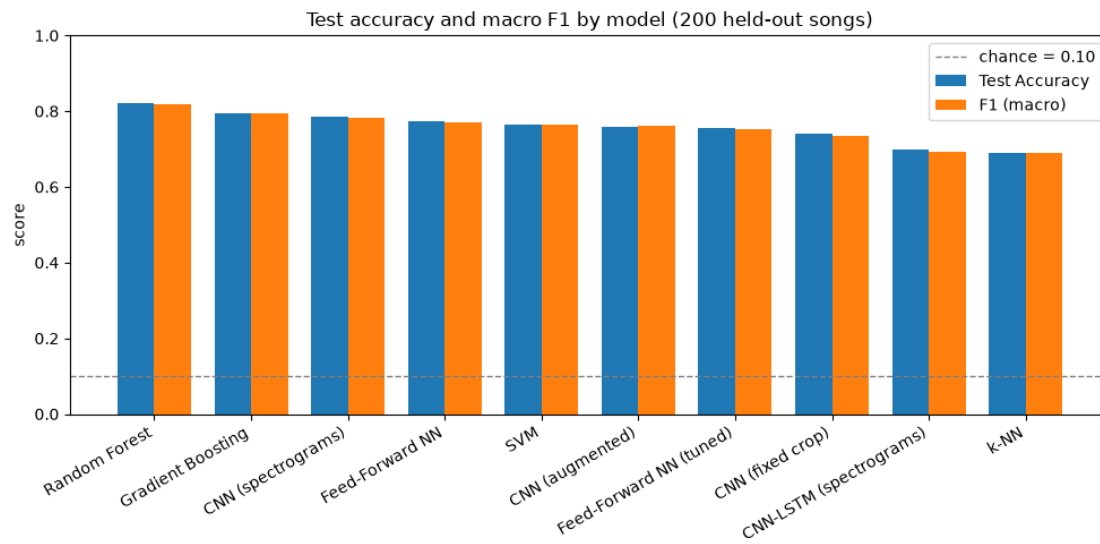
13. Test Cases

The notebooks include explicit checks, each written to confirm or rule out a specific failure.

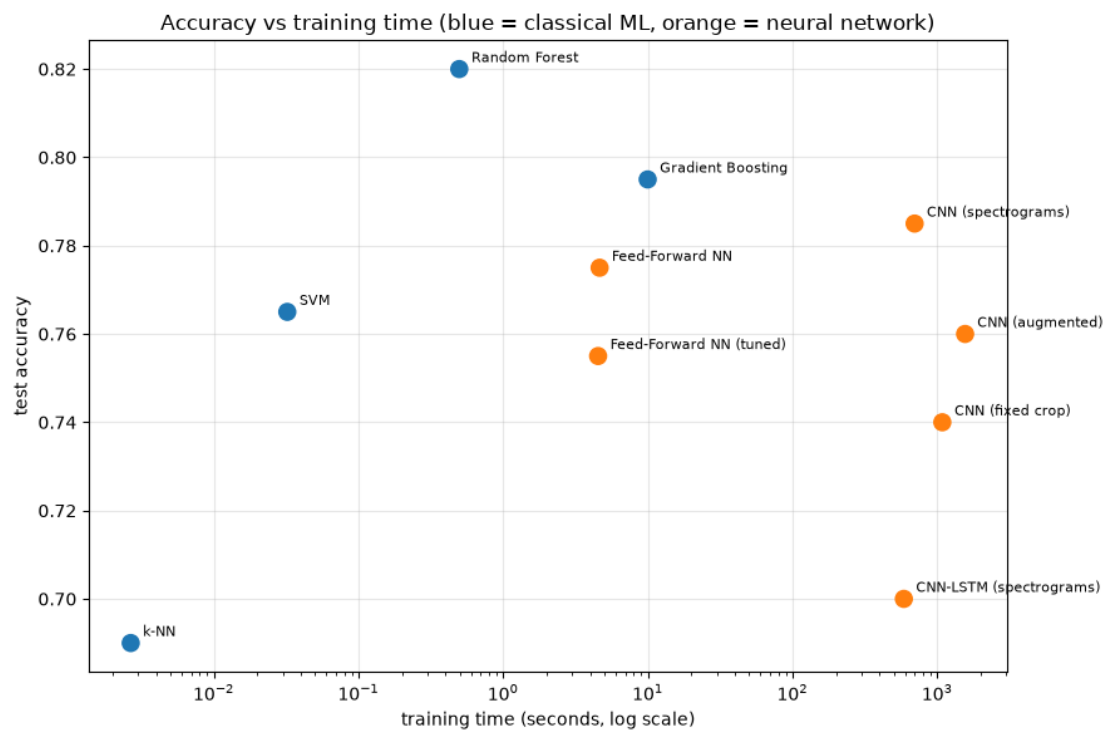
#	Test	Expected	Result
1	Class balance of the split	80 training and 20 test songs per genre	Confirmed for all ten genres
2	Spectrogram split matches the feature split	799 training images, 200 test, one song without an image	799 / 200 / 1 (the missing clip is jazz)
3	Validation split is stratified	16 validation songs per genre	[16 16 16 16 16 16 16 16 16 16]
4	CNN has not collapsed to one class after the batch-normalization fix	predictions spread across all genres	[15 19 17 13 13 13 16 16 16 22]
5	Original augmentation varies within one pass	non-zero differences between draws	[0.0, 0.070, 0.070]
6	Corrected augmentation varies within and across passes	non-zero differences both ways	within: [0.0, 0.074, 0.063]; across: 0.067
7	Training is deterministic for a fixed configuration	tuning sweep reproduces the Section 2.2.3 run	best epoch 29, validation accuracy 0.73125 in both
8	Tuning winner identified correctly	winner equals the builder's default configuration	winner_is_the_baseline = True
9	Sample predictions, one song per genre	illustrative only	baseline CNN 5 of 10 correct; augmented CNN 7 of 10

Test 9 is illustrative only: with ten songs the standard error is about 13 points, so the two samples should not be compared.

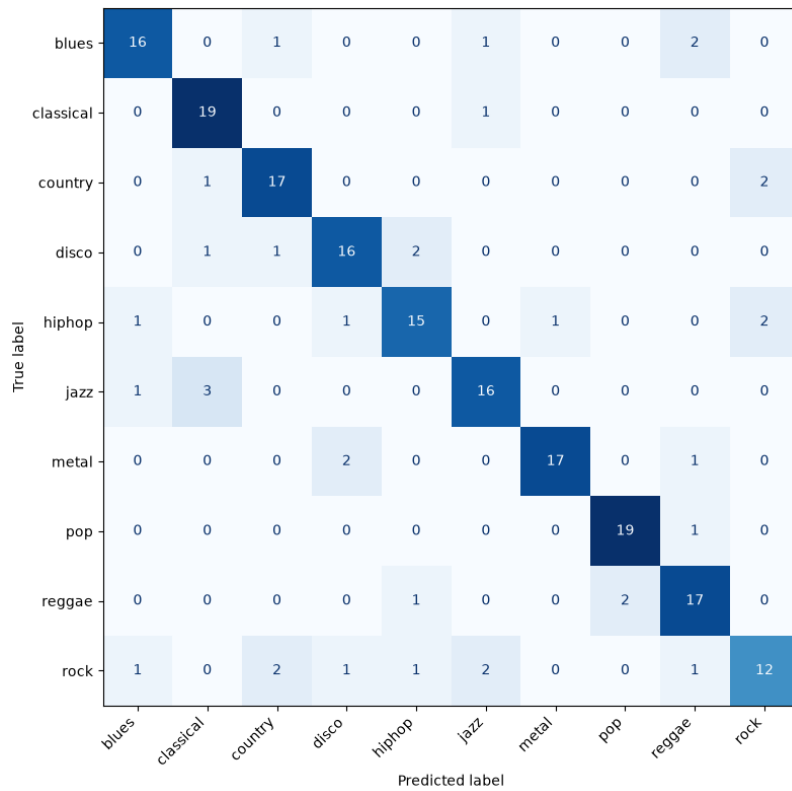
14. Graphs, Plots and Performance Figures



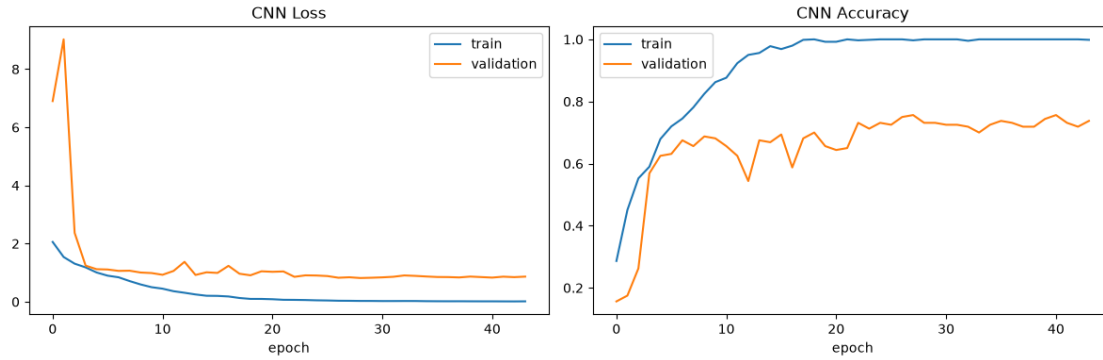
Test accuracy and macro F1 for all ten models on the 200 held-out songs. The dashed line is the 10% chance rate.



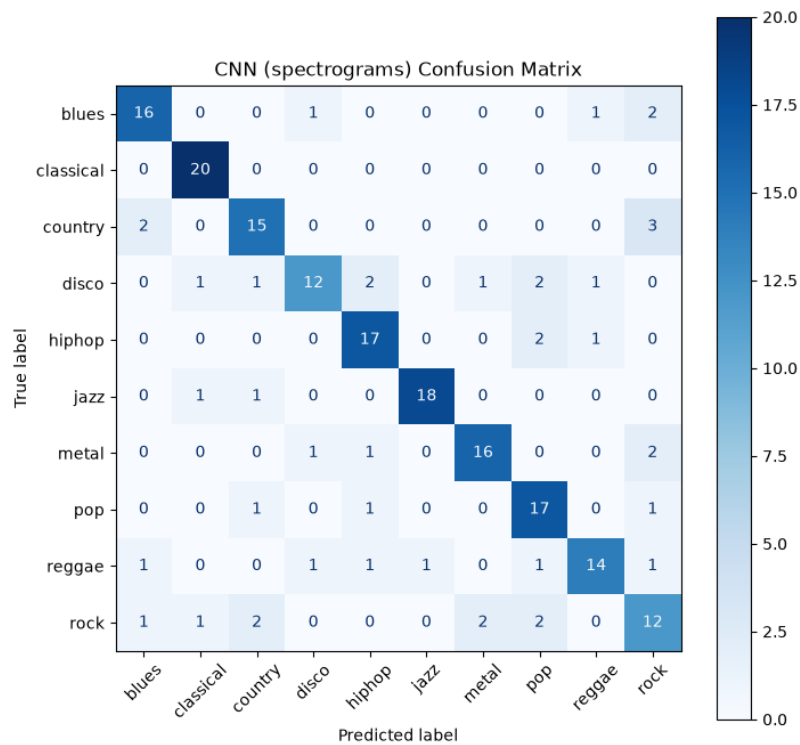
Test accuracy against training time (log scale). Classical models in blue, neural networks in orange. Random Forest is both the most accurate and among the fastest.



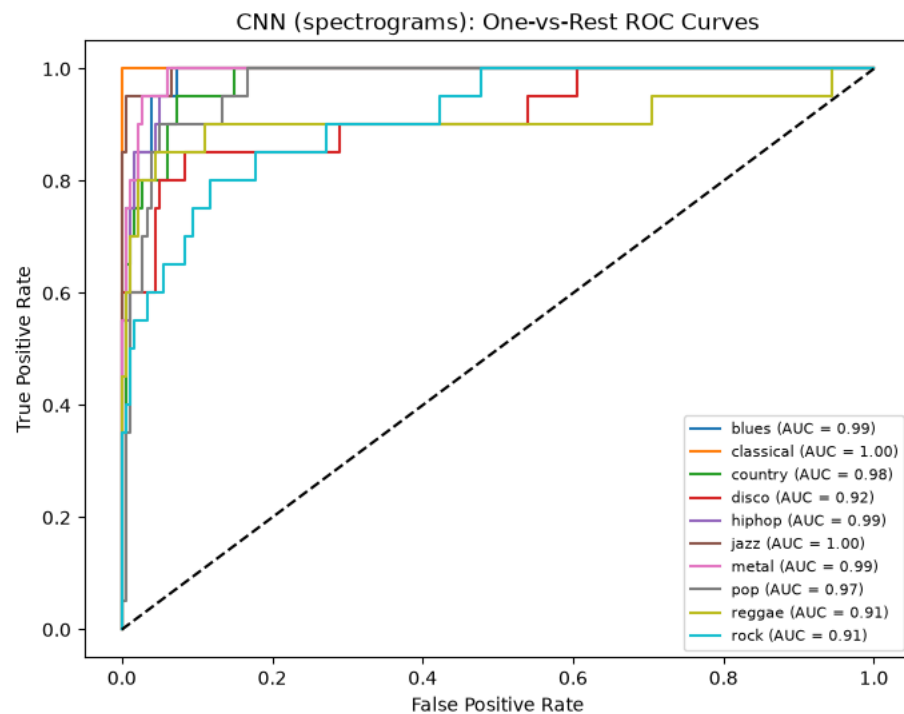
Tuned Random Forest confusion matrix — the best model in the project.



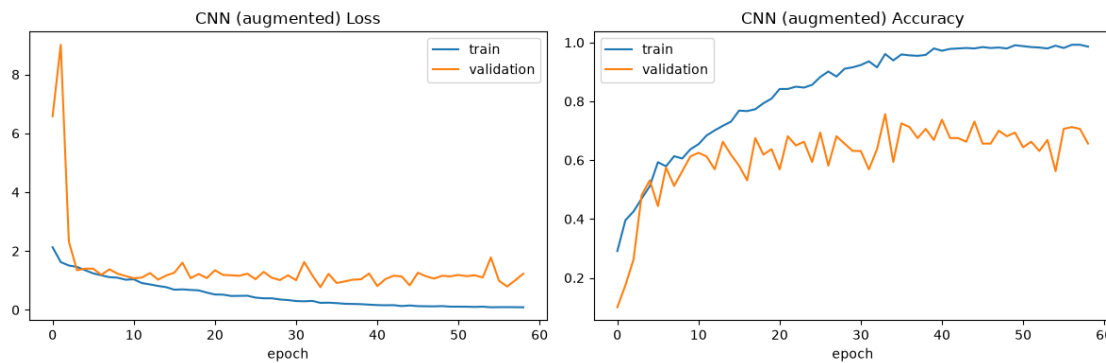
Baseline CNN learning curves. Training accuracy reaches essentially 1.00 by about epoch 18 while validation accuracy plateaus near 0.74 — the signature of memorization.



Baseline CNN confusion matrix. Classical is identified perfectly; rock and disco are the weakest genres.



Baseline CNN one-vs-rest ROC curves. Rock's AUC of 0.91 is far above its 0.60 recall: the correct genre is usually ranked highly but not first.



Augmented CNN learning curves. Augmentation slowed memorization but left validation accuracy noisier and no higher.

15. Optimization Techniques Used

- **Grid search with cross-validation** for all four classical models — 329 hyper-parameter combinations in total, each scored by 5-fold cross-validation on the training set.
- **Manual hyper-parameter sweeps** for the neural networks: five configurations for the feed-forward network and a 2×2 grid for the CNN, each scored on a fixed stratified validation set.
- **Early stopping with best-weight restoration** for every network, which selects the number of training epochs on validation loss — itself the most important hyper-parameter for models that reach near-perfect training accuracy.
- **Dropout** (0.3) in every network.
- **Batch normalization** in the convolutional models, with momentum tuned to 0.9 so its running statistics converge on a small dataset.
- **Data augmentation** — time-axis cropping, resizing and masking of spectrograms.
- **Feature standardisation** for all feature-table models.
- **Reduced-budget tuning** for the CNN (40 epochs, patience 8, instead of 100 and 15) to keep the sweep to about 26 minutes. This was a deliberate trade-off; one configuration reached the 40-epoch cap, so its score is a lower bound.

16. Standards Considered, Constraints and Limitations

16.1 Standards and good practice

- **No test-set leakage:** the test set was never used for training, tuning or model selection; the feature scaler was fitted on training data only.
- **Song-level splitting:** each song appears on exactly one side of the split in both data representations.
- **Stratification:** the train/test and train/validation splits both preserve the exact genre balance.

- **Reproducibility:** a fixed random seed (42) for every split and every model; all results produced by top-to-bottom runs of the notebooks from a restarted kernel.
- **Consistent evaluation:** every model scored by the same metrics, on the same songs, in the same results schema.

16.2 Constraints

- **Compute:** all training ran on CPU. Individual CNN runs took 10–26 minutes, the CNN tuning sweep 26 minutes, the Gradient Boosting grid search 54 minutes, and a full top-to-bottom run of the Part 2 notebook about 1.5 hours.
- **Data volume:** 1,000 clips in total, of which 639 were available to train the convolutional networks after the test and validation sets were held out.
- **Pre-computed inputs:** the audio features and spectrogram images were supplied with the dataset, which fixed their resolution and content.
- **Time:** Course ran from July through early September. I also had a few trips during this time period that limited my availability to work on the project on some days.

16.3 Limitations

- **Single split, no repeated evaluation.** Every figure rests on one 800/200 split; with 200 test songs each accuracy carries a 95% margin of about ± 5.3 points, and a single genre's recall moves by 0.05 for every song.
- **Run-to-run variation.** The augmented CNN runs are not bit-reproducible, because the `tf.data` pipeline consumes random numbers in a non-deterministic order. Three full runs gave test accuracies between 0.74 and 0.775 for identical code — a direct measure of the noise underlying every small difference in the results table.
- **Dataset faults.** GTZAN is known to contain repeated clips, mislabelled clips and repeated artists, which can inflate measured accuracy [Sturm (2013) — see References].
- **Fixed input representations.** The spectrograms are pre-rendered PNG plots rather than spectrograms computed from the audio, which fixes their time-frequency resolution and colour mapping.

17. Challenges Faced and How They Were Addressed

1. **The first CNN behaved like a random classifier.** Its test accuracy was near the 10% chance rate even though training accuracy rose normally. Comparing predictions in inference mode against training mode isolated the cause: batch normalization's running mean and variance. With only about 20 weight updates per epoch (639 images at batch size 32), the Keras default momentum of 0.99 left those statistics essentially untrained, so inference collapsed. Setting momentum to 0.9 fixed it, and a follow-up check confirmed predictions spread across all ten genres.
2. **Keras's validation split silently used the wrong songs.** The image arrays are ordered by genre, and `validation_split` takes the last 20% without shuffling — it

would have validated on only the last genre or two. An explicit stratified 639/160 split replaced it.

3. **Aligning two data representations.** The feature table names files `blues.00000.wav` while the images are named `blues00000.png`. Re-using the identical split call on the filename column and converting the names reproduced the Part 1 split exactly for the images; one clip (jazz) turned out to have no image.
4. **Augmentation that did not augment.** The first augmentation function drew its random numbers in a way that reset for every pass through the data, so every epoch replayed the identical augmented images. A first diagnostic test was itself flawed and reported no variation for the wrong reason; a corrected test separated variation *within* a pass from variation *across* passes and located the bug. Rewriting the function around a seeded `tf.random.Generator` fixed it, verified by the same tests.
5. **Results that would not reproduce.** After the fix, repeated runs of the augmentation experiments produced different numbers each time, and the comparison between the two augmentation variants reversed between runs. Removing parallelism from the data pipeline was tried and was not sufficient. Rather than report a single run as definitive, the variation was measured across three runs and reported as a finding in its own right.
6. **Stale results in a long notebook.** Re-running cells out of order produced six different recorded training times for the same CNN and several narrative claims that no longer matched the output. This was addressed by restarting the kernel, running both notebooks top to bottom, and re-checking every number in the written analysis against the final outputs.
7. **A hypothesis that did not survive.** One run showed disco absorbing songs from tempo-adjacent genres, which fitted a plausible explanation — that time-stretching augmentation blurs tempo cues. The next run did not reproduce the pattern, so the explanation was withdrawn and recorded as an unsupported hypothesis.

18. Conclusion and Future Work

18.1 Conclusion

Ten models were compared on the same 200 held-out songs. The best was a tuned Random Forest at 0.820 accuracy, trained in half a second; the best neural network, a CNN on spectrograms, reached 0.785 after more than a thousand times as much training time. With a ± 5.3 -point margin on every estimate, the top five models are not statistically separable, and only k-NN and the CNN-LSTM are clearly worse.

The consistent message is that **model capacity was not the limiting factor — data volume was**. Hyper-parameter tuning, data augmentation and a larger architecture each failed to improve on simpler baselines, and every high-capacity model memorized its training set with a generalization gap of around 20 points. The per-genre pattern — classical easy, rock hard and over-predicted, country confused with blues and rock — was

stable across models and families, pointing to genuine overlap between genres in the data rather than to any one algorithm's weakness.

These findings agree with published work: Akhtar and Mishra (2025) also found that a classifier trained on engineered audio features outperformed a CNN on GTZAN, attributing it to CNN overfitting on a moderately sized dataset. By contrast, Boyina et al. (2022) report accuracies above 90% on the same dataset, but using 3-second segments where segments of the same song can fall on both sides of the train/test split — a comparison that illustrates why the song-level split used here gives lower but more defensible numbers.

18.2 Future work

- **Use the 3-second segment features** (9,990 rows instead of 999) with a strictly song-level split, to test directly whether more training examples close the gap.
- **Transfer learning** from a pretrained image or audio network — the standard remedy for training convolutional networks on small datasets.
- **Soft-voting ensembles** of the existing models, which require no retraining and would show whether their errors are correlated.
- **Repeated or cross-validated evaluation** of the neural networks to put confidence intervals on every result.
- **Reproducible augmentation** using stateless, per-example random seeds.
- **A logistic-regression baseline**, to show how much of the signal is linearly separable.

19. Ethics and Bias

- **Genre labels are subjective.** Genre boundaries are culturally constructed and contested; many songs legitimately belong to more than one. A single-label classifier forces a choice that listeners themselves would dispute.
- **One small dataset cannot represent the world's music.** GTZAN holds 1,000 clips in ten Western popular genres, dating from around the turn of the millennium. The world's music is vastly larger and more varied than that, and a model trained on this sample has no representation of most of it: it would force a Brazilian forró, an Indian film song or a Ghanaian highlife track into whichever of its ten Western labels looks nearest.
- **Who is represented is unknown, and probably skewed.** The dataset records only a genre label for each clip — nothing about the performers. Their gender, race and ethnicity, country of origin, language, socio-economic background, wealth, sexual orientation and political views are neither recorded nor balanced, so the dataset cannot even be audited for these biases, let alone corrected for them. What can be said is that datasets of this kind are assembled from music that is easy to obtain, and recent Western music from English-speaking countries is far more heavily represented in commercial catalogues and online than music from lower-income

countries and the Global South. A classifier inherits that skew: it will be most reliable for the music that is already most visible, and least reliable for the music that most needs discovering.

- **What mitigation would require.** A system intended for real use would need a far larger and deliberately diverse dataset, metadata on performers' origin and language so that coverage could be measured, and evaluation reported separately for each group rather than as a single overall accuracy.
- **Label errors become model errors.** GTZAN contains documented repetitions and mislabellings [Sturm (2013)]. Any such faults are learned by the model and also distort its measured accuracy.
- **Systematic bias in this project's own models.** The baseline CNN predicts "rock" 21 times for 20 rock songs, only 12 of them correctly — it is the label the model reaches for when uncertain. In a recommendation or cataloguing system, that behaviour would push songs from other genres into rock and reduce the visibility of those genres and their artists. Reporting per-genre precision and recall, not only overall accuracy, is what exposes this.
- **Overstated performance is itself a harm.** Evaluation protocols that let segments of the same song appear in both training and test data can report accuracies far higher than a system would achieve on new music. Deploying a model on the strength of such numbers would mislead its users.
- **Copyright.** The clips are excerpts and used here for educational research. Copyright / licensing concerns could prevent the resulting trained models from being used in commercial software.

References

1. GTZAN Dataset — Music Genre Classification. Kaggle.
<https://www.kaggle.com/datasets/andradaolteanu/gtzan-dataset-music-genre-classification>
2. Tzanetakis, G. and Cook, P. (2002). Musical genre classification of audio signals. *IEEE Transactions on Speech and Audio Processing*
3. Akhtar, R. and Mishra, A. (2025). *Music Genre Classification Using Machine Learning Techniques*. arXiv:2509.01762. Bhagwan Parshuram Institute of Technology, New Delhi. <https://arxiv.org/html/2509.01762v1>
4. Boyina, A., Dahiya, R., Chandan, S. and Gupta, A. (2022). Music Genre Classification. *Medium*, 8 January 2022.
<https://medium.com/@ashitaboyina/music-genre-classification-70ae70469403> — code: github.com/ashcode028/Music-Genre-Classification
5. Sturm, B. L. (2013). The GTZAN dataset: its contents, its faults, their effects on evaluation, and its future use. <https://arxiv.org/pdf/1306.1461>
6. course materials, python modules, scikit-learn and TensorFlow documentation

Appendix — Project Files

- Brian Miller - Final Project - Part 1.ipynb — classical machine learning models
- Brian Miller - Final Project - Part 2 - Neural Networks.ipynb — neural networks and the combined comparison
- part1_results.csv, all_results.csv — exported results tables